



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Tècnica Superior d'Enginyeria
de Telecomunicació de Barcelona



Work-flux optimization for the use of GRASP algorithm in quasi-real-time

Master Thesis
submitted to the Faculty of the
Escola Tècnica d'Enginyeria de Telecomunicació de Barcelona
Universitat Politècnica de Catalunya
by
Mireia López Barrón

In partial fulfillment
of the requirements for the master in
(*Write the name of your Master*) **ENGINEERING**

Advisor: name of the advisor
Barcelona, Date XXXXX



Contents

List of Figures	4
List of Tables	4
1 Introduction	7
1.1 Gantt Diagram	7
1.2 Deviation from the initial plan	8
2 State of the art	9
2.1 GRASP algorithm	9
2.1.1 Input data	9
2.2 Current execution flow	10
2.2.1 Download data	11
2.2.2 Select configuration and execute Matlab scripts	15
2.2.3 Modify manually configuration file for GRASP algorithm	16
2.2.4 Execute GRASP algorithm in CALCULA	16
2.2.5 Execute Matlab script for postprocessing and results analysis	16
2.2.6 Need of optimisation	17
3 Methodology	18
4 List of requirements	19
5 Research, plan and adapt	21
5.1 Web services development	21
5.1.1 ACTRIS–EARLINET service	21
5.1.2 AERONET service	22
5.2 Matlab controller development	24
5.2.1 Create script configuration files and output files	24
5.2.2 Automate MATLAB repository for needed data files	24
5.2.3 Adapt old scripts to new file formats	24
5.2.4 Execute MATLAB scripts from C#	24
6 Architecture	26
6.1 MVVM	26
6.1.1 Observable Object	27
6.1.2 Relay Command	27
6.2 SOLID principles	28
6.2.1 Single Responsibility Principle	28
6.2.2 Open/Closed Principle	28
6.2.3 Liskov Substitution Principle	28
6.2.4 Interface Segregation Principle	29
6.2.5 Dependency Inversion Principle	29
6.3 Project architecture	29

7	Graphic User Interface (GUI) design	31
7.1	Download data files	32
7.2	Data combination	33
7.3	Plot GRASP results	34
8	Implementation	36
8.1	Set up development enviroment	36
8.2	Views and ViewModels	37
8.3	Controlllers	38
8.4	Polymorphism through interfaces	39
8.4.1	Factory pattern	39
8.4.2	Project Actions	40
8.4.3	Download Controlllers	41
8.4.4	Matlab Scripts Implementations	41
8.5	Interactions with other applications	42
9	Testing	44
10	Results	46
10.1	Final User Interface	46
10.2	Work-flux	49
10.3	Low resolution User Interface	49
11	Conclusions	52
	References	53
	Appendices	55

List of Figures

1	Project's Gantt diagram	8
2	Earlinet website for downloading data	11
3	Aeronet website for downloading data	12
4	Aeronet site selection	13
5	Aerosol Optical Depth download UI	14
6	Aerosol Inversions download UI	15
7	GRASP configuration file	16
8	Project Development Cycle	18
9	MMV design pattern	26
10	MVVM design pattern applied to the project architecture	29
11	TabMenu design	31
12	Home design	32
13	Download tab design	33
14	Data combination tab design	34
15	Project folders organisation	34
16	Plot tab design	35
17	extensions	36
18	Application interactions diagram	43
19	Implemented Unit Tests	45
20	Home view	46
21	Download view	47
22	Data Combination view	47
23	Plotting view	48
24	Settings window	48
25	Low resolution UI	49
26	Plotting view	50
27	Download view for low resolution UI	50
28	Data Combination view for low resolution UI	51
29	Plotting view for low resolution UI	51

Listings

List of Tables

1	AERONET Data Products and Extensions	22
---	--	----

Revision history and approval record

Revision	Date	Purpose
0	10/12/2025	Document creation
1	dd/mm/yyyy	Document revision

DOCUMENT DISTRIBUTION LIST

Name	e-mail
[Student name]	
[Project Supervisor 1]	
[Project Supervisor 2]	

Written by:		Reviewed and approved by:	
Date	dd/mm/yyyy	Date	dd/mm/yyyy
Name	Xxxxxxxx yyyyyyy	Name	Zzzzzzzz Wwwwwww
Position	Project Author	Position	Project Supervisor

Abstract

Every copy of the thesis must have an abstract. An abstract must provide a concise summary of the thesis. In style, the abstract should be a miniature version of the thesis: short introduction, a summary of the results, conclusions or main arguments presented in the thesis. The abstract may not exceed 150 words for a Degree's thesis.

1 Introduction

Recently, the need to analyze and monitor atmospheric aerosols has grown significantly, as their negative impact has grown significantly in different sectors: human health, climatology and air transport. GRASP (Generalized Retrieval of Atmosphere and Surface Properties), which was introduced by Dubovik, is the first unified algorithm used for obtaining atmospheric properties from different remote sensing observations including satellite, ground-based and airborne measurements, both passive and active, of atmospheric radiation and their combinations[25].

The main objective of this work is to develop a software that automates the full GRASP execution process, which includes:

1. Download photometer data from the Aeronet web site
2. Download Lidar (Light Detection and Ranging) data from the ACTRIS-EARLINET web site
3. Combine the data downloaded from both web sites
4. Run GRASP
5. Plot the obtained results

1.1 Gantt Diagram

The time organization of the project is shown in the Gantt diagram of Figure 1 representing a time lapse of one semester. It is important to note that even there are different groups of tasks for implementation and testing, tests have been executed during the implementation phase to ensure the correct functionality of the different steps.

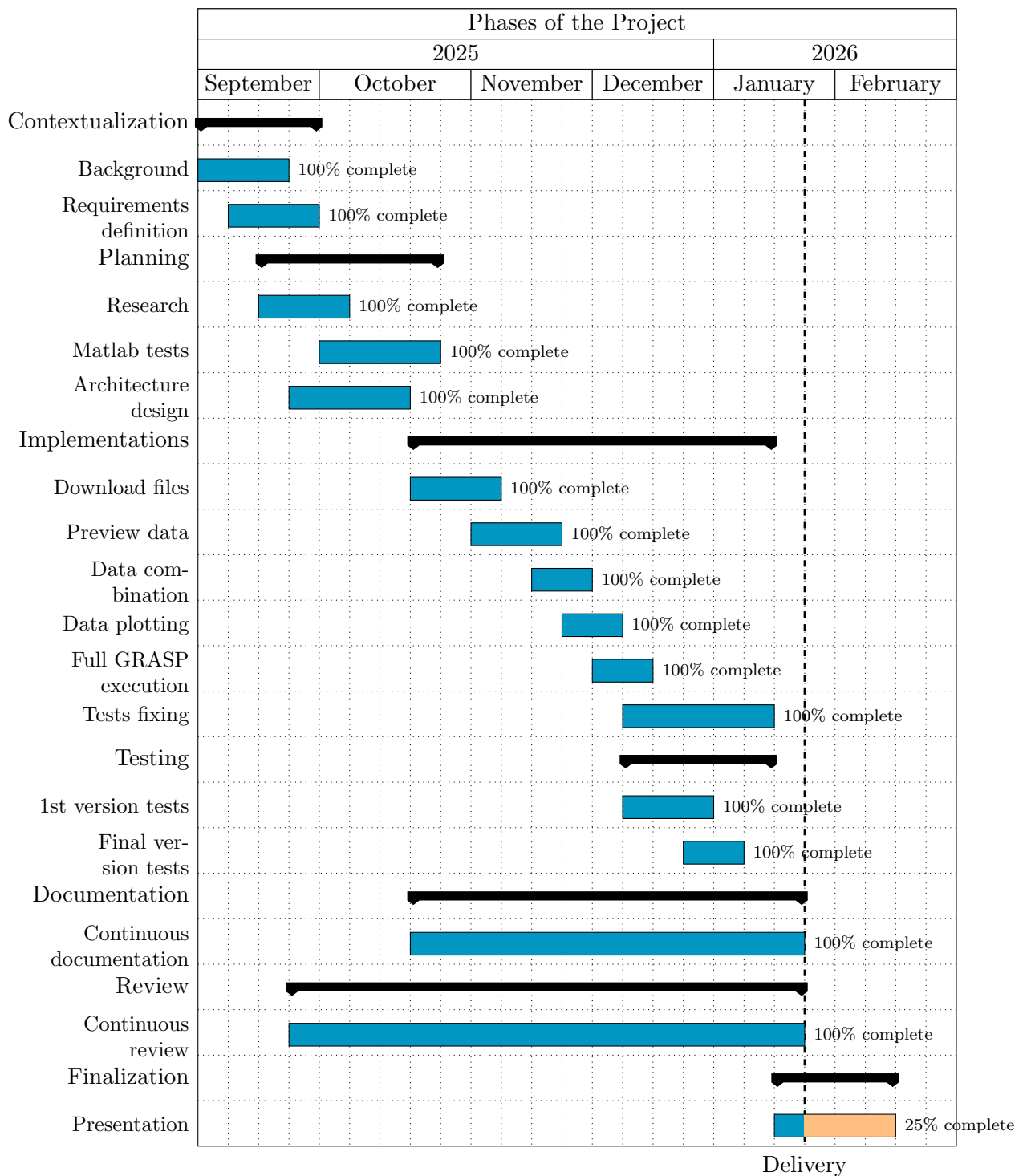


Figure 1: Gantt diagram of the project

1.2 Deviation from the initial plan

2 State of the art

2.1 GRASP algorithm

GRASP (Generalized Retrieval of Aerosol and Surface Properties) is a unified and highly accurate algorithm used to study the atmosphere. It retrieves many different aerosol and surface characteristics from remote-sensing observations. According to the GRASP-OPEN website, the algorithm can estimate **nearly 50 parameters**, such as particle-size distribution, refractive index, sphericity, and absorption. It is designed to work with **spectral, multi-angle, and polarimetric** measurements, and it performs well even over **bright surfaces** like deserts, where the ground reflectance usually hides the aerosol signal.

It combines information from different instruments—both satellite and ground-based—to produce a detailed description of aerosols and surface reflectance. Its main features include:

- Processing reflectance data from aerosols and land surfaces to infer physical and optical properties.
- Using multi-angle and polarimetric observations, which provide more information than simple intensity measurements.
- Integrating diverse data sources, making it adaptable to many observation systems.
- Applying a unified inversion approach, meaning it uses the same mathematical framework for all types of input data. This helps ensure consistency and accuracy across different instruments [25].

This algorithm is valuable because it offers a comprehensive and consistent way to analyse atmospheric aerosols. These particles affect climate, air quality, and visibility, so understanding them is essential. GRASP's ability to work with many types of measurements makes it suitable for:

- Climate and environmental studies
- Air-quality monitoring
- Satellite-data validation
- Research on aerosol–surface interactions

2.1.1 Input data

GRASP algorithm aims to analyse different types of measurements in order to The UPC lidar data used in GRASP come from the Single Calculus Chain (SCC), an online system for processing and storing lidar measurements. The lidar information includes:

- Elastic backscatter at 355, 532, and 1064 nm: these channels record the total backscattered signal at each wavelength. Although the laser emits linearly polarized light, these channels do not separate polarization. The signals are taken from the SCC's ELPP module and pre-processed by:

- applying distance correction,
- resampling to 60 logarithmic points,
- normalising the profile.

This data is mandatory in order to run the GRASP algorithm.

- Depolarized backscatter at 355 and 532 nm: these channels detect only the component of the signal perpendicular to the transmitted polarization. GRASP cannot use these signals directly, so the volume depolarization must be calculated. This is done in the SCC's ELDA module.

The application developed in this project automatically checks which lidar measurements are available for each date and identifies whether total-power and/or depolarization data exist at the required wavelengths.

2.1.1.1 Photometer data

The CIMEL CE318 photometer collects different types of measurements, organized in “scenarios,” which are automatically sent to the AERONET server and later used as input for GRASP. The main scenarios include:

- Aerosol Optical Depth (AOD): a direct measurement from sunlight or moonlight, available both during the day and on moonlit nights. The result is adjusted to represent a vertical atmospheric column. This data is mandatory in order to run the GRASP algorithm.
- Almucantar scans: the instrument measures diffuse irradiance while rotating in azimuth at a fixed solar elevation. This data is mandatory in order to run the GRASP algorithm.
- Polarized almucantar scans: similar to the previous case, but measuring the degree of linear polarization (DoLP) introduced by aerosols.
- Main-plane scans: measurements taken by sweeping in elevation while keeping the solar azimuth fixed, for both total irradiance and DoLP.
- Hybrid scenarios: combinations of the above.

GRASP can currently combine AOD, total-irradiance almucantar data, DoLP, and lidar measurements. Only a limited number of photometers worldwide provide polarized almucantar data, so the system must check whether these measurements are available for each date.

During calibration periods, the replacement photometer may not record DoLP, which affects the available inputs.

2.2 Current execution flow

Universitat Politècnica de Catalunya (UPC) research department for Remote Sensing has been using the GRASP algorithm to process and obtain information from data measured

by a lidar and a photometer, in order to study the presence of aerosols in the atmosphere. In order to do that, different steps need to be executed manually, without a graphical interface or centralized application, which increases the risk of errors, reduces reproducibility, and demands significant user intervention.

Other students have previously developed Matlab scripts in order to automate the pre-processing stage of the workflow used by the department. However, this solution is not optimal as it requires the user to have a basic knowledge of Matlab and all the different web sites and files used.

Every time someone wants to use the GRASP algorithm, they need to follow the steps described in the different points of this section.

2.2.1 Download data

2.2.1.1 Earlinet

First website to download data is Actris-Earlinet. This interface is used to download the data measured by the Lidar, and can be easily downloaded by configuring different parameters like start and finish date, and selecting the desired station, as it can be seen in Figure 2. The different files that can be downloaded will appear listed under the configuration parameters list.

The screenshot shows the ACTRIS-EARLINET Data Portal interface. On the left, there are navigation buttons for 'SEARCH DATA', 'DATASETS', and 'QUICKLOOKS'. The main area contains search filters: 'Date Minimum (YYYY-MM-DD)' set to '2026-01-04', 'Date Maximum (YYYY-MM-DD)' set to '2026-01-11', 'Stations' set to 'BRC', 'Emission Wavelengths', 'File Types', and 'Levels'. Below these is an 'ADVANCED SEARCH' section with a 'SEARCH' button. The 'RESULT' section displays a table of data files with columns: File Name, Ty, Start, Stop, Sta, EV, Date U, Lc, and a set of icons for file actions. The table lists several files named 'EARLINET_AerRemSen_brc_Lev01_b0355_20z' with various start and stop times. At the bottom, there are logos for 're3data.org', 'ACTRIS', 'EARLINET', and the European Union flag, along with a copyright notice: 'v2.1.0 - © C.N.R. IMAA 2024 - All rights reserved.'

Figure 2: Earlinet website for downloading data

From this website, it is always downloaded Range Cross Signal in wavelengths 355, 532 and 1064 nm. There are exception periods, where 1064 nm signal is not downloaded. In this case, GRASP cannot be executed.

Volume Depolarization can be also downloaded, but it is not always present in the downloaded measurements in some periods of time. Even if this data is missing, GRASP can still be executed without problem by using the correct configuration.

2.2.1.2 Aeronet

Aeronet website offers more information and options to download data, so it needs to be done in different steps. First, the user needs to locate the Aeronet Data Access page, as it can be seen in Figure 3, and select a data option.

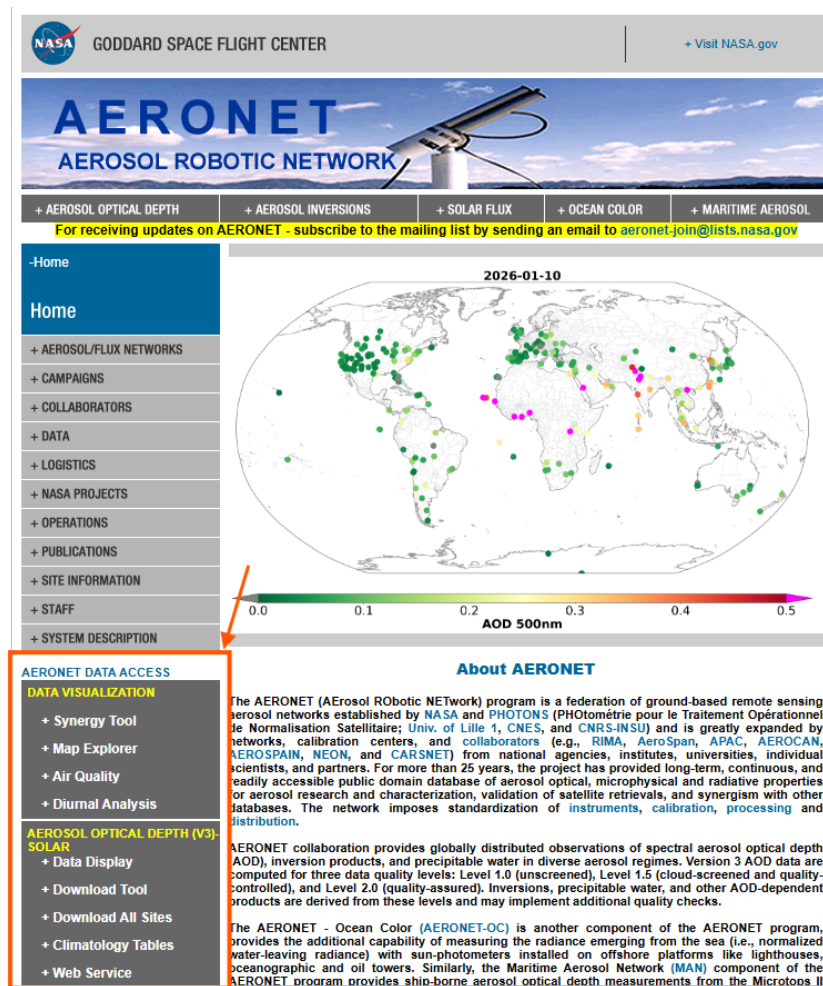


Figure 3: Aeronet website for downloading data

For each type of data, the user needs to select the desired site from the shown list, as it can be seen in Figure 4.

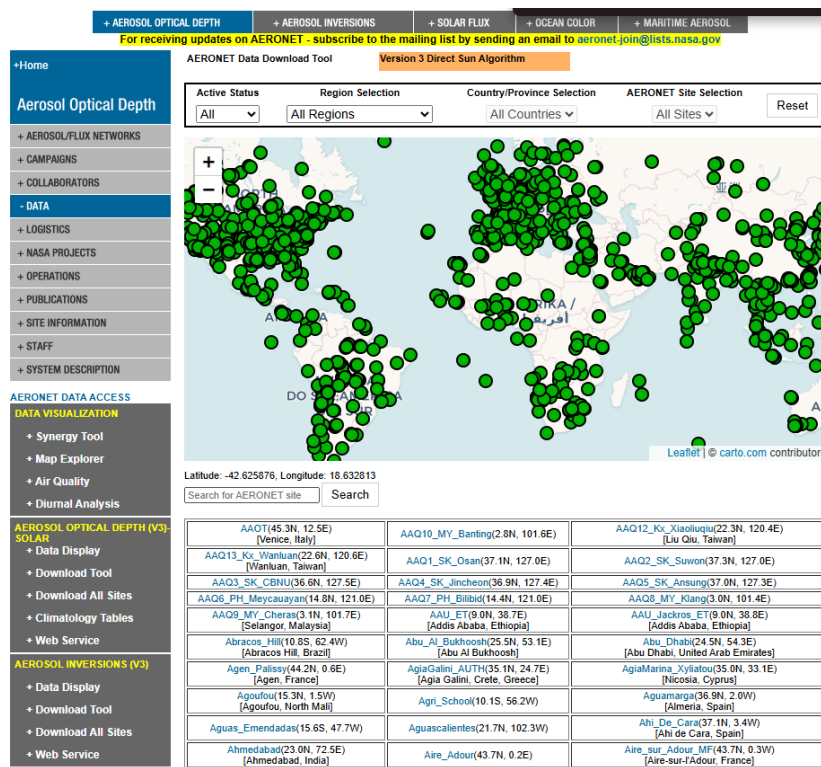


Figure 4: Aeronet site selection

Once the site is selected, the user needs to choose the different data options to be downloaded for both, AOD and Inversions, as it can be seen in Figure 5 and Figure 6, respectively.

GODDARD SPACE FLIGHT CENTER
[+ Visit NASA.gov](#)

AERONET

AEROSOL ROBOTIC NETWORK

+ AEROSOL OPTICAL DEPTH
+ AEROSOL INVERSIONS
+ SOLAR FLUX
+ OCEAN COLOR
+ MARITIME AEROSOL

+ Home
Aerosol Optical Depth
+ AEROSOL/FLUX NETWORKS
+ CAMPAIGNS
+ COLLABORATORS
- DATA
+ LOGISTICS
+ NASA PROJECTS
+ OPERATIONS
+ PUBLICATIONS
+ SITE INFORMATION
+ STAFF
+ SYSTEM DESCRIPTION

AERONET Data Download Tool
Version 3 Direct Sun Algorithm

Click Geographic Region, Country/State or AERONET Site to change site selection:

Geographic Region
Europe
Country/State
Spain
AERONET Site
Barcelona

Download Data for Barcelona
Select the start and end time of the data download period:

START: Day/Month/Year
1 JAN 2004
END: Day/Month/Year
31 DEC 2026

Data Descriptions
Data Units

Note: Data are not available if the data type is *italicized*

Select the data type(s) using the corresponding check box:

Direct Sun Products	Select
Aerosol Optical Depth (AOD) with Precipitable Water and Angstrom Parameter	Level 1.0 ☐ Level 1.5 ☒ Level 2.0 ☐
Total Optical Depth based on AOD Level*	Level 1.0 ☐ Level 1.5 ☒ Level 2.0 ☐
Spectral Deconvolution Algorithm (SDA) Retrievals -- Fine Mode AOD, Coarse Mode AOD, and Fine Mode Fraction	Level 1.0 ☐ Level 1.5 ☒ Level 2.0 ☐

Data Format

☒ All Points
☐ Daily Averages
☐ Monthly Averages

Download

*All Points Format Only

Figure 5: Aerosol Optical Depth download UI

+ CAMPAIGNS
+ COLLABORATORS
- DATA
+ LOGISTICS
+ NASA PROJECTS
+ OPERATIONS
+ PUBLICATIONS
+ SITE INFORMATION
+ STAFF
+ SYSTEM DESCRIPTION

AERONET DATA ACCESS
DATA VISUALIZATION
+ Synergy Tool
+ Map Explorer
+ Air Quality
+ Diurnal Analysis
AEROSOL OPTICAL DEPTH (V3)- SOLAR
+ Data Display
+ Download Tool
+ Download All Sites
+ Climatology Tables
+ Web Service
AEROSOL INVERSIONS (V3)
+ Data Display
+ Download Tool
+ Download All Sites
+ Web Service
SOLAR FLUX
+ Data Display
OCEAN COLOR
+ V3 Data Display
+ V3 Web Service
+ Download All Sites
LUNAR AOD (V3)

EuropeSpainBarcelona

Download Data for Barcelona

Select the start and end time of the data download period:

START: Day/Month/Year
1 JAN 2004

END: Day/Month/Year
31 DEC 2026

Data DescriptionsData Units

Note: Data are not available if the data type is *italicized*

Select the data type(s) using the corresponding check box:

Radiance Products (with calibration and temperature correction applied)**		Select
Raw Almuqantar	☒	
Raw Hybrid	☐	
Raw Principal Plane	☐	
Raw Polarized Principal Plane (with degree of polarization)	☐	
Raw Polarized Almuqantar (with degree of polarization)	☒	
Raw Polarized Hybrid (with degree of polarization)	☐	

**Available using All Points option

Derived Inversion Products		Select
Uncertainty Estimates*	☐	
Spectral Flux	☐	
Radiative Forcing	☐	
Volume Size Distribution	☐	
Size Distribution Parameters	☐	
Refractive Index	☐	
Coincident AOD	☐	
Single Scattering Albedo	☐	
Absorption AOD	☐	
Extinction AOD	☐	
Lidar and Depolarization Ratios	☐	
Asymmetry Factor	☐	
Phase Function	☐	
All Products Except Phase Function and U27	☒	

Product Scenario and Data Quality

Sky Scan Scenario: ☒ Almuqantar ☐ Hybrid

Data Quality: ☐ Level 1.5 ☒ Level 2.0

Data Format

☒ All Points ☐ Daily Averages ☐ Monthly Averages

Download

Figure 6: Aerosol Inversions download UI

It can also be downloaded the inversions results from the AERONET website ??, which it is used for comparing the results after executing the GRASP algorithm.

Once all data has been downloaded, it needs to be located in a file accessible for the Matlab scripts.

2.2.2 Select configuration and execute Matlab scripts

There are different configurations, depending on the available data downloaded:

- D1L: Only photometer and Lidar data are available
- D1P_L: It is also available polarization data in photometer measurements
- D1L_VD: It is also available depolarization data in Lidar measurements
- D1P_L_VD: All data is available

Knowing the available files and the selected dates, in order to run the Matlab Scripts successfully, the user should manually choose the configuration and execute the corresponding

script, since there is one script per configuration possibility.

2.2.3 Modify manually configuration file for GRASP algorithm

After executing the Matlab scripts, a data file is created named `jmeasureIDj_GARRLIC_jconfigurationj.s` which will be used as input for the GRASP algorithm. But a configuration file is also needed, that indicates different parameters to the algorithm, being one of them the name of the data file to be used. This file is named `UPC_jconfigurationj.yml`.

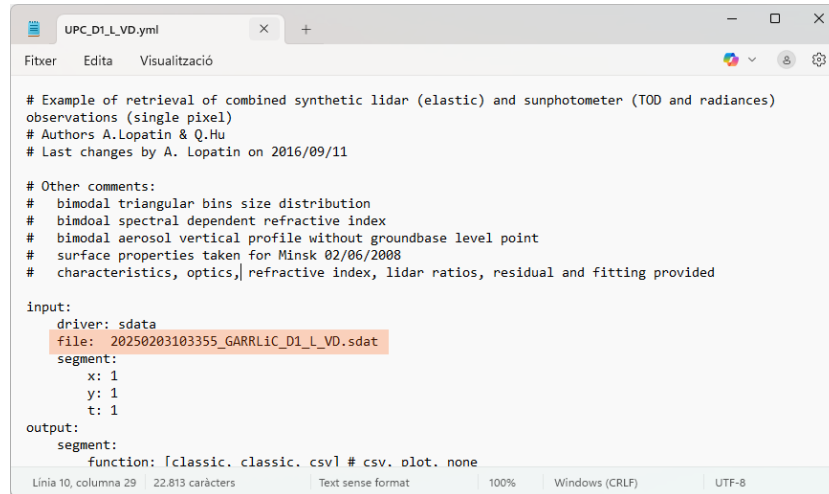


Figure 7: Example of GRASP configuration file

In Figure 7, it can be seen the example of a configuration file when the configuration chosen is D1L_VD and the MeasureID is 20250203103355, corresponding to the 3rd of February at 10:33:55.

2.2.4 Execute GRASP algorithm in CALCULA

When all required files have been created and correctly configured, the GRASP algorithm can be executed in CALCULA. The user should upload the files to the virtual environment and execute the following command lines in the command prompt:

```

cd <directory to files>
/opt/bin/grasp-1.1.5/grasp UPC<config>.yml
  
```

Once the algorithm has been executed correctly, files named `UPC_jconfigurationj_screen.txt` and `UPC_jconfigurationj_out.txt` will be created in same directory.

2.2.5 Execute Matlab script for postprocessing and results analysis

After the execution, the user needs to download the output files from CALCULA and execute the Matlab post processing script in order to obtain the results and the corresponding figures to analyse.

2.2.6 Need of optimisation

After analysing the process, it can be seen that it is not optimal as it requires the user to access different websites in order to download the needed files, analyse the available data in order to select the correct configuration and execute scripts manually, without a graphical interface or centralised application, which increases the risk of errors, reduces reproducibility, and demands significant user intervention.

Working with different Operating Systems (mostly Windows and CALCULA) also increases the time needed to execute the process.

The main objective of this project is to optimise all of the steps of this sequence so the time consumption and the error introduction possibilities by the user are reduced.

3 Methodology

In order to create a new application, it is necessary to follow the following steps, to create a strong foundation.

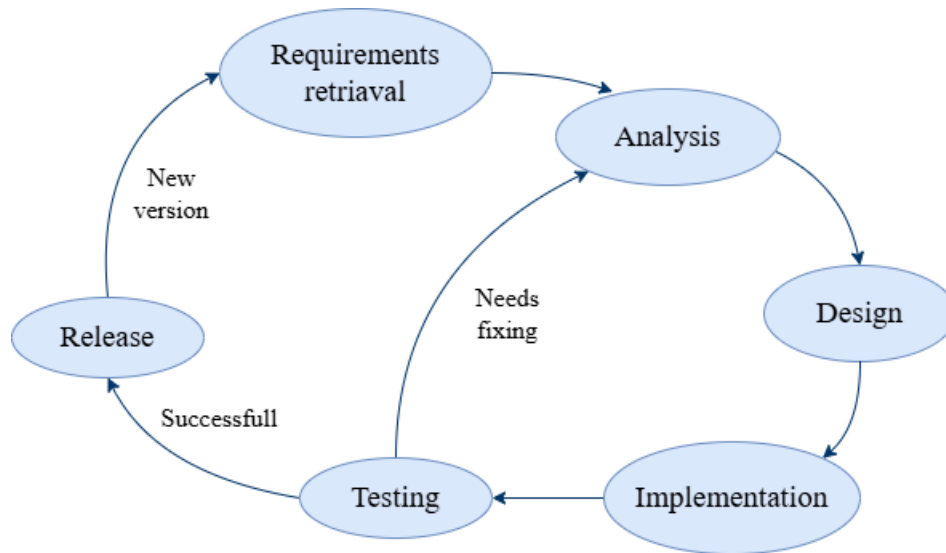


Figure 8: Project development cycle

This diagram shows the ideal development process for a project. Since the application developed is the first version, requirements retrieval should only happen once, and the project should finish with the Release phase, once all tests have been passed correctly.

For the development of the application, it has been decided to use a hybrid structure, using Object Oriented Programming (OOP) with C# and MATLAB scripting.

The main advantages of using C# with OOP are that it allows the code to be reusable, organised and easy to maintain [7]. In this project, it has been used for the development of the Graphical User Interface and the controllers for both, web services and MATLAB script management.

On the other hand, MATLAB provides a specialised environment for numerical calculations, simulations, and complex data analysis[8]. Working directly with matrices and arrays makes the data processing and visualisation easier and more efficient.

4 List of requirements

This first step is crucial for the correct result. It is needed to define a clear and well defined list of requirements. This defines the main scope of the project, the different futures to be implemented and prevents from building something that is not desired. This project, need to be able to fulfill the following requirements:

1. Application can be executed in CALCULA operating system (Linux)
2. Download measurements from the internet
 - (a) Download measurements from a defined date range
 - (b) Download measurements from a defined location
 - (c) Download measurements from the Actris-Earlinet Data Portal [5]
 - Download ELPP products
 - Download Optical products
 - (d) Download measurements from the Aeronet web [6]
 - Download Raw Almucantar Sky Scan Radiance measurements
 - Download Raw Hybrid Sky Scan Radiance measurements
 - Download Raw Principal Plane Sky Scan Radiance measurements
 - Download Raw Polarized Principal Plane Sky Scan Radiance and Degree of Polarization measurements
 - Download Raw Polarized Almucantar Sky Scan Radiance and Degree of Polarization measurements
 - Download Raw Polarized Hybrid Sky Scan Radiance and Degree of Polarization measurements
3. Filter Earlinet measurement files depending on the the data type
 - (a) 002, 008
 - (b) 007
4. Execute pre-execution:
 - (a) Read data of all downloaded files that are used as inputs for grasp algorithm
 - (b) Preview data in order to choose a correct value for minimum and maximum heights
 - (c) Obtain available configurations for the selected measure ID depending on available data
 - (d) Choose a configuration
 - (e) Generate a .sdat file that stores all data for selected measure ID and heights

-
- (f) Create .yaml configuration file containing corresponding .sdat file name and chosen configuration
 5. Execute the GRASP algorithm with corresponding configuration .yaml file and .sdat file
 6. Give the user the possibility to plot the different results after executing the algorithm
 - Detect all available measure IDs folders in output directory
 - Generate figure data files .mat from GRASP output data in selected measure ID output directory
 - Plot data from .mat files
 7. Application can work with project/workspaces
 - (a) User can create, open and import a project
 - (b) Downloaded data from web services are stored in the project folder
 - (c) Results of the GRASP algorithm are stored in the project folder
 8. Application can have different configurations
 - Repository directories can be modified for both, earlinet and aeronet downloaded files
 - Output directory can be modified
 - GRASP installation directory can be modified: this path can be different for each user

5 Research, plan and adapt

This step helps optimise both time and resources by preventing unnecessary work. Since this project builds upon a previous one, it is not required to develop new code for certain application requirements, but to adapt it to the new needs. Furthermore, the availability of APIs for downloading measurements from the ACTRIS-EARLINET Data Portal [5] and AERONET web application [6] provides a convenient and efficient solution that can significantly reduce development effort.

5.1 Web services development

5.1.1 ACTRIS-EARLINET service

The ACTRIS-EARLINET Data Portal provides a REST API for downloading measurements [9]. This website shows all the different requests that are supported by the API, gives examples of how to use them and the expected response.

Having all this information available is really helpful, since it allows to localise the requests that are needed for the project requirements.

After testing and comparing the different options and the obtained responses, it becomes clear that the best option is to use the `"/products/downloads"` endpoint, which allows to download the measurements in a compressed format, filtering the data by type, date and station name.

Even if other configuration parameters are available (measurementID, wavelength and opticaltype), it is not necessary to use them for the project requirements.

Once the behaviour is understood, a class is created to handle the requests and responses of the API.

```
1 public class EarlinetService{
2     string baseUrl = "https://api.actris-ares.eu/api/services/restapi/";
3     public virtual System.Threading.Tasks.Task<bool>
4         DownloadProductByDateRangeAsync(string type, string fromDate,
5         string toDate, string outputFolder){
6         return DownloadProductByDateRangeAsync(type, fromDate, toDate,
7         outputFolder, System.Threading.CancellationToken.None);
8     }
9
10    public async Task<bool> DownloadProductByDateRangeAsync(string type,
11    string fromDate, string toDate, string outputFilePath, System.
12    Threading.CancellationToken cancellationToken = default){
13        try{
14            string url = $"{baseUrl}products/downloads?kind={type}&
15            fromDate={fromDate}&toDate={toDate}&stations=brc";
16            using (HttpClient client = new HttpClient()){
17                HttpResponseMessage response = await client.GetAsync(url);
18            };
19            response.EnsureSuccessStatusCode();
20            using (var fs = new FileStream(outputFilePath, FileMode.
21            Create, FileAccess.Write, FileShare.None)){
```

```

14         await response.Content.CopyToAsync(fs);
15     }
16     System.IO.FileInfo f = new FileInfo(outputFilePath);
17     if (f.Length <= 1048)
18         return false;
19     return true;
20 }
21 }
22 catch (Exception ex){
23     return false;
24 }
25 }
26 }

```

For the creation of this class it has been taken into account the fact that the DownloadProductByDateRangeAsync method can be used in different points of the program with different configurations. It has also been implemented in such a way that adding new functionalities is easy using the same design pattern.

5.1.2 AERONET service

The AERONET web does not provide a REST API portlet, but it includes web service documentation that helps the implementation of a custom service.

The amount of files to be downloaded in this website is larger than the one in Earlinet, and its documentation can be found in three different sections of the web: For the

Table 1: AERONET Data Products and Extensions

Category	Product Description	Extension
Optical Depth [10]	Aerosol Optical Depth (AOD): (descripcion de los datos)	.lev15
	Spectral Deconvolution Algorithm (SDA): (descripcion de los datos)	.ONEILL.lev15
Aerosol Inversions [12]	Inversion products: (descripcion de los datos)	.all
Raw Products Optical Depth [11]	Raw Almucantar: (descripcion de los datos)	.alm
	Raw Polarized Almucantar: (descripcion de los datos)	.alp

creation of this class it has been taken into account the fact that the DownloadProductByDateRangeAsync method can be used in different points of the program with different configurations. It has also been implemented in such a way that adding new functionalities is easy using the same design pattern. For clarity and organisation, the DataType Enum

has been created, since the different data types need different Url content in order to be downloaded.

```
1 public class AeronetService{
2     private string baseUrl = "https://aeronet.gsfc.nasa.gov/cgi-bin/";
3     public async Task DownloadDataAsync(string destinationFile, string
4         url){
5         try{
6             using (HttpClient client = new HttpClient()){
7                 var response = await client.GetAsync(url);
8                 response.EnsureSuccessStatusCode();
9
10                var content = await response.Content.
11                    ReadAsByteArrayAsync();
12                await File.WriteAllBytesAsync(destinationFile, content);
13            }
14        }
15        catch (Exception ex){
16            Console.WriteLine($"Error downloading data: {ex.Message}");
17        }
18    }
19
20    public string BuildUrl(DataType _dataType, DateTime startDate,
21        DateTime endDate, string productType = "", string site = "", string
22        product = "", string AVG = "", bool isEnabled = false){
23        switch (_dataType){
24            case DataType.AerosolInversions:
25                if (isEnabled)
26                    return BuildUrlAerosolInversions(startDate, endDate,
27                        productType, site, product, AVG);
28                else
29                    return BuildUrlAerosolInversions(startDate, endDate);
30
31            case DataType.OpticalDepth:
32                if (isEnabled)
33                    return BuildUrlOpticalDepth(startDate, endDate,
34                        productType, site, AVG);
35                else
36                    return BuildUrlOpticalDepth(startDate, endDate,
37                        productType);
38
39            case DataType.RawProductsOpticalDepth:
40                if (isEnabled)
41                    return BuildUrlRawProductsOpticalDepth(startDate,
42                        endDate, productType, site, AVG);
43                else
44                    return BuildUrlRawProductsOpticalDepth(startDate,
45                        endDate, productType);
46        }
47        return "";
48    }
49
50    {...} //BuildUrl methods can be found in GitHub repository
51 }
52 }
```

In this application, this class is only used to download the data files listed above, but other types could be downloaded by calling only the `DownloadDataAsync` and `BuildUrl` methods in a line.

5.2 Matlab controller development

As it was mentioned in the introduction, this project aims to automate an execution process of a set of scripts in MATLAB.

The previously developed code has been adapted to the new hybrid system in order to be executed with C#. In order to do that, different actions have been taken.

5.2.1 Create script configuration files and output files

Since the project is hybrid, it is necessary to create a configuration file for the script that will be executed in C#. This file will contain the parameters needed to execute the script. In the previous approach, these files were not needed, since GUI introduced parameters were being stored in the MATLAB workspace. The use of these files allows the communication between the two different languages. Configuration files allow the different scripts to obtain the information and parameters that the user introduced, while the output files allow the application to know the corresponding results and notify the user about it.

5.2.2 Automate MATLAB repository for needed data files

Previously, it was necessary to download manually all the data files needed, then, when Matlab Scripts were executed, a dialog window would appear asking for the location of each file. For different data, a different amount of files were needed, having a maximum of five files to select. In order to automate this process, a script has been created that downloads the data files from the AERONET website and stores them in a local repository. In order to be able to automate the process of downloading and giving the application the capacity to work with different workspaces, the repository location has been modified and automated inside the application. The corresponding file is saved as configuration parameter in both, application and script.

5.2.3 Adapt old scripts to new file formats

5.2.4 Execute MATLAB scripts from C#

In order to execute the MATLAB scripts from C#, it is needed to create a method that uses the `ProcessStartInfo` class, that can be imported as a library, to execute the MATLAB script. The following example shows a method called `RunMatlabScript` that takes a string parameter containing the path to the script to be executed. The method uses the `Process` class to execute the script and it redirects the standard output and standard error to the console.


```

1 public static void RunMatlabScript (string scriptPath){
2     try{
3         ProcessStartInfo startInfo = new ProcessStartInfo{
4             FileName = "matlab",
5             Arguments = $"-batch_\\"run('{scriptPath}')\\"",
6             RedirectStandardOutput = true,
7             RedirectStandardError = true,
8             UseShellExecute = false,
9             CreateNoWindow = true
10        };
11        using var process = new Process { StartInfo = startInfo };
12        process.OutputDataReceived += (s,e) =>{
13            if (e.Data != null)
14                Messenger.Default.Send<string>("WriteMatlabOutput",e.
15                    Data);
16        };
17        process.ErrorDataReceived += (s,e) =>{
18            if (e.Data != null)
19                Messenger.Default.Send<string>("WriteMatlabErrors",e.
20                    Data);
21        };
22        process.Start();
23        process.BeginOutputReadLine();
24        process.BeginErrorReadLine();
25        process.WaitForExit();
26        if (process.ExitCode == 0){ //OK
27            Logger.Log($"Script was executed correctly, executing post_
28                execution_actions.");
29            script.PostExecutionActions();
30        }
31        else{
32            Logger.Log($"Error occurred during execution, executing post_
33                execution_actions.");
34            script.PostExecutionActions(false);
35        }
36    }
37    catch (Exception ex){
38        Logger.Log($"Error occurred during execution, executing post_
39            execution_actions.");
40        script.PostExecutionActions(false);
41    }
42 }

```

6 Architecture

6.1 MVVM

Design pattern MVVM (Model-View-ViewModel) is a software architecture pattern designed with the goal of having a clear separation between the user interface (frontend) and the business logic (backend) [13]. There are different frameworks that use this standard, such as Angular, .NET WPF...

This application needs to be able to execute in CALCULA operating system (Linux), so it is necessary to use a framework that is compatible with this system. Originally, it was going to be developed using .NET WPF, but it was decided to use Avalonia instead, as it is a cross-platform framework that can be executed in different operating systems. The differences between both frameworks are very low, mainly being the name of the files (.axaml instead of .xaml) and the way to define the user interface.

This architecture is divided in three main components:

- **Model:** It contains the data and most of the app logic. It structures the information (classes and identities), how to retrieve it and how to manage it (services, data bases and controllers).
- **View:** It defines all the elements in the user interface and what the user will see and will interact with. The only responsibility of the code defines in this sections is to define the visual structure of the app and to create input elements for the user to interact with.
- **ViewModel:** It is the bridge between the Model and the View. It is the one that controls the flow of data between the Model and the View.

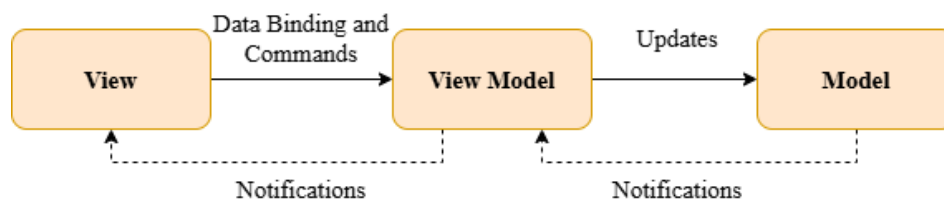


Figure 9: MVVM design pattern

The most important advantages of using this design pattern are:

- Changes only affect the view or the model, not both
- Separated testing for views and models
- User interface can be modified without affecting the model
- Teams with different developers can work on the same project simultaneously without affecting each other

6.1.1 Observable Object

In order to be able to update the user interface automatically when the data changes, it is used Data Binding, a feature of the MVVM pattern. It is implemented using ObservableObject as base class.

```
1 public class User : ObservableObject
2 {
3     private string name;
4
5     public string Name
6     {
7         get => name;
8         set => SetProperty(ref name, value);
9     }
10 }
```

In the example, it can be seen how a variable is updated, when it is detected that the new value is different from the old one, SetProperty is called, which implements INotifyPropertyChanged interface, which is the one in charge to notify the corresponding view.

In order that the view is updated, it is necessary to call the variable with the same name and to indicate that it is Bindable in the corresponding .axaml file, as it can be seen in the following example:

```
1 <TextBlock Text="{Binding Name, Mode=TwoWay, UpdateSourceTrigger=
    PropertyChanged}" />
```

In this case, mode option indicate that variable can be updated from the view to the model and viceversa, and update source trigger option indicate that the update will be done when the property changes. Other options can be used, but these are the ones that are most common and that have been used in this project.

6.1.2 Relay Command

Relay command is used to implement commands in the MVVM pattern. It is implemented using ICommand as base class. It can be used in different ways, but in this project it has been implemented using the following pattern for all View Models:

```
1 public class ViewModel : ObservableObject
2 {
3     public ICommand Cmd => new RelayCommand(Execute, CanExecute);
4     private void Execute(object _)
5     {
6         ExecutionActions();
7     }
8
9     private bool CanExecute(object _)
10    {
11        return true;
12    }
13 }
```

In order to execute the command, it is necessary to link an interface element of the .axaml file to the Execute method

```
1 <Button Content="Execute" Command="{Binding_Cmd}" />
```

6.2 SOLID principles

SOLID principles are a set of five principles that are used to design software with the objective of making it easy to maintain and modify, making a project more maintainable and scalable [16]. It lowers the dependency between classes, minimizing the impact of changes and making the code more reusable, maintainable, flexible and stable.

They also support growth in development, making it easier to add new features without the need of modifying already implemented code.

These principles are:

- Single Responsibility Principle (SRP)
- Open/Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)

6.2.1 Single Responsibility Principle

States that *a class should have only one reason to change*, meaning that it should have only one responsibility, job or purpose within the software project.

In order to apply this principle, it is necessary to divide the code into different classes, each one with a specific responsibility. It is important to note that this principle is not always possible to apply, but it is a good practice to try to do so.

6.2.2 Open/Closed Principle

States that *an entity should be open for extension, but closed for modification*, meaning that it should be easy to extend the functionality of the entity without modifying its existing code.

In order to apply this principle it can be used abstraction by implementing Interfaces and Abstract classes and by using different design patterns like Strategy or Decorator.

6.2.3 Liskov Substitution Principle

Introduced in 1987 by Barbara Liskov, it states that *derived or child classes must be substitutable for their base or parent classes*, ensuring that any class can be used in place of its parent class without any loss of functionality.

In order to apply this principle, it is important to ensure that subclasses can be used anywhere where the base class is used without changing the expected behaviour, avoiding

overriding methods and making sure to not throw `NotImplementedException`. This can be done by using Interfaces and Abstract classes.

6.2.4 Interface Segregation Principle

This principle applies directly to interfaces, and it states that *do not force any client to implement an interface which is irrelevant to them*. The main objective of this principle is to not implement too big interfaces and to implement instead smaller and more specific interfaces.

If this principle is used in a project, the corresponding software architecture should have many interfaces implementations with specific purposes.

6.2.5 Dependency Inversion Principle

States that *high-level modules should not depend on low-level modules, but both should depend on abstractions*, ensuring that the dependencies are inverted and that the high-level modules are not dependent on the low-level modules.

In order to apply this principle, it is suggested to use abstractions and interfaces to define the dependencies between modules.

6.3 Project architecture

Applying the MVVM design pattern and the SOLID principles, the project architecture can be summed up as follows:

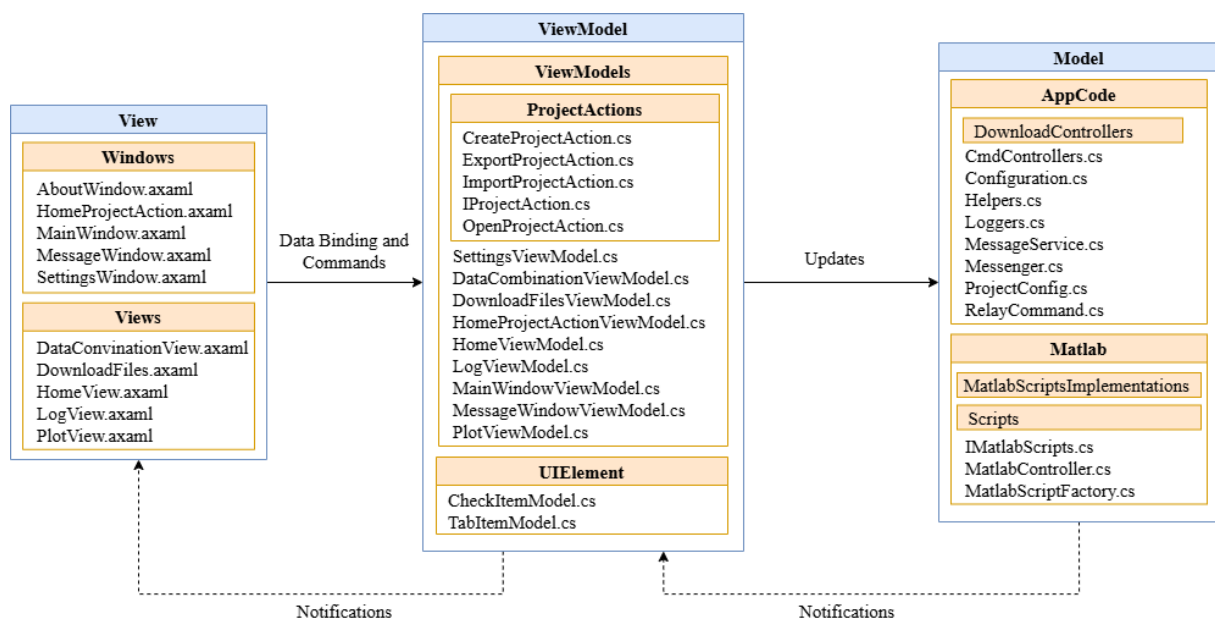


Figure 10: MVVM design pattern applied to the project architecture

In Figure 10 it can be seen how the Views are divided into different folders. The Windows folder contains all the files containing the definitions of the different windows and dialogs

of the application, while the Views folder contains all the files containing the definitions of different UserControls. UserControls are used as Content in the MainWindow, in this case, as Contents of the different tab items.

The ViewModel section contains all the files containing the definitions of the different ViewModels of the application. Most of them are located in the ViewModels folder, but it can also be appreciated that there is a folder named UIElement, where the models for CheckItems and TabItems are located. With this classes, adding these items to Views and ViewModels is easier to maintain and update.

Finally, the Model component is composed by all the internal classes of the application. It has been divided by AppCode, where is located all the implementations related to app Logic, and by Matlab, where all the MATLAB scripts are located aswell as the necessary implementations for the app to execute them.

7 Graphic User Interface (GUI) design

In order to give to the user only the essential information, it was chosen to divide the different main actions that the application should perform into different views. In order to accomplish that, different UI implementations can be used. In this case, it was decided to implement a TabMenu, with three different views for each action: Download data files, Combine Data for GRASP algorithm execution and Plot GRASP results.

In this section, the design of the GUI will be described. A prototype of each view will be shown. In Figure 11, the corresponding view for the MainWindow can be seen.

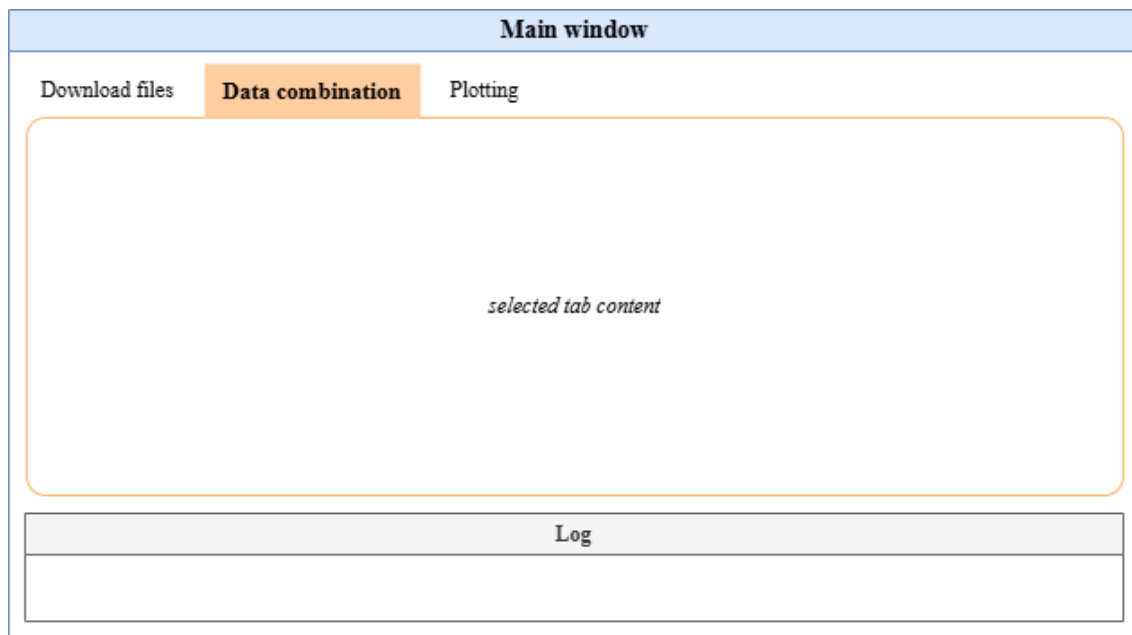


Figure 11: TabMenu design

With this window, the user will be able to access all the different functionalities of the application, which should be executed in the following order:

1. Download data files
2. Combine Data for GRASP algorithm execution
3. Plot GRASP results

Other necessary Windows were for Configuration, About and Message Windows. The content of these windows should be as simple as possible, since they are complementary windows and they do not contain a lot of information. For the Configurations window, it is only necessary to have one input element in order to introduce the value for each configuration parameter. Design can be modified taking into account the necessity of the current application.

When the application is first initialised, a Home View is needed, with different buttons,

where the user can choose which type of action they want to perform: Create, Open or Import project.

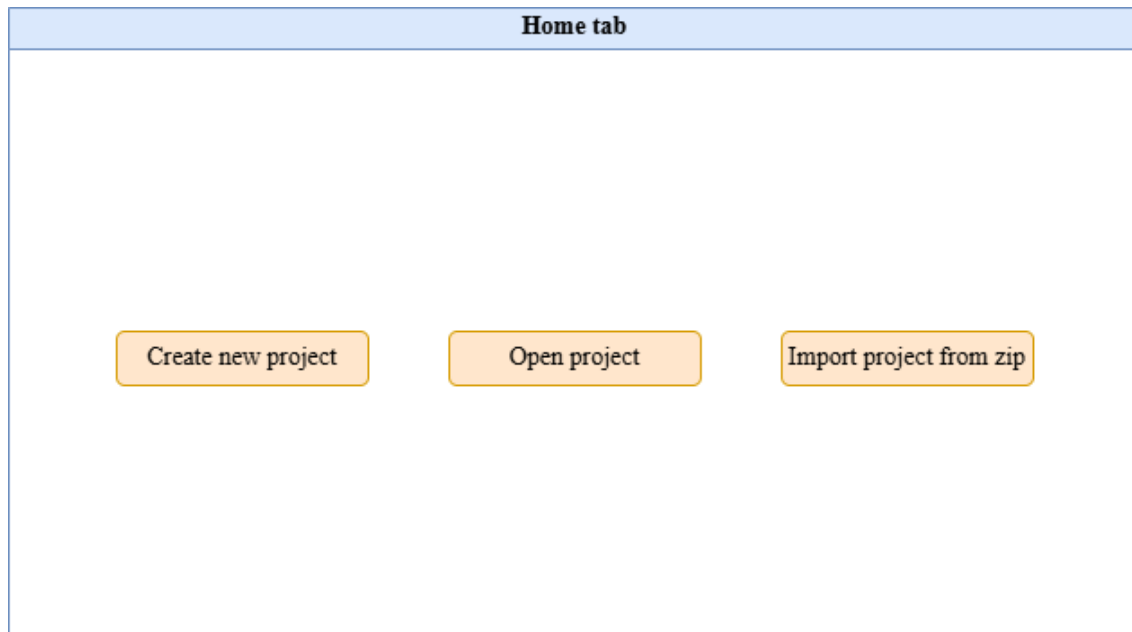


Figure 12: Home design

7.1 Download data files

In the requirements list, it was specified that the user should be able to download the data files within a selected date range and a selected station. After that, the user should be able to see a list of the downloaded files from each website. In order to implement this functionality, the first design uses two calendar selectors in order to choose "From date" and "To date". A text selector is used in order to select the station and a Download Button in order to start the execution. To finish, a list of the downloaded files is shown in a text block. All these features can be seen in the Figure 13.

Download files tab

Calendar (from date)

Calendar (to date)

Downloaded files list

File 1
File 2
File 3

Selected station

Download button

Figure 13: Download tab design

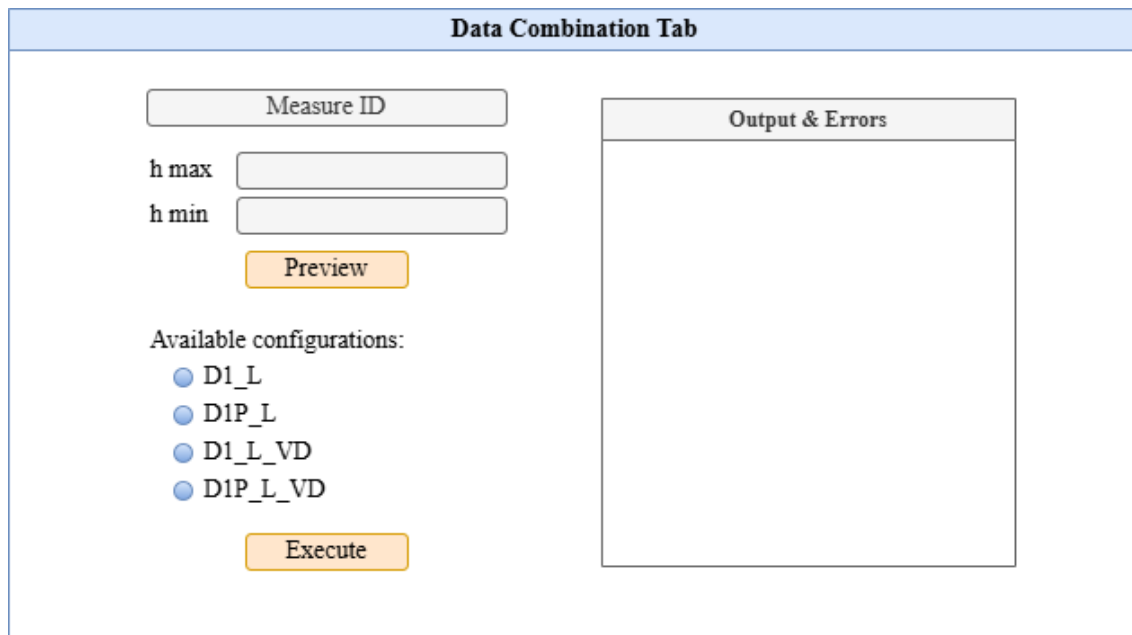
7.2 Data combination

For the data combination step, the user needs to select with which of the available measure IDs the GRASP algorithm is going to be executed. It is also necessary to have two options in order to select the minimum and maximum height for the data combination.

Once the data is previewed, it is necessary to notify the user of the different configurations that are available so that the user can choose one of them depending on the available data for the selected date and time.

Two buttons are used in order to start the different scripts execution: one for the data preview and the other one for generating the combination .sdatt and .yaml files and executing the GRASP algorithm execution.

All these features can be seen in Figure 14. In addition, a text section should be added in order to see the Output and Errors obtained from executing the different Matlab scripts used for pre-processing the data. Thanks to that element, the user will be able to see the process during the execution and the errors that may occur during it.



The figure shows a software interface titled "Data Combination Tab". It is divided into two main sections. The left section contains a "Measure ID" input field, followed by "h max" and "h min" input fields. Below these is a "Preview" button. Further down, it says "Available configurations:" followed by four radio button options: "D1_L", "D1P_L", "D1_L_VD", and "D1P_L_VD". At the bottom of this section is an "Execute" button. The right section is titled "Output & Errors" and contains a large empty rectangular box for displaying results.

Figure 14: Data combination tab design

7.3 Plot GRASP results

For the last tab, the plotting tab, it is necessary to understand how the project folders are organised in order to know all the necessary elements that are going to be needed so that the user can introduce all the necessary information. As it can be appreciated in Figure 15, the project folder is organised in different subfolders, each one with a specific purpose. Data folder will contain all the downloaded data and Output data will contain the results of both, the data combination script and the GRASP algorithm execution.

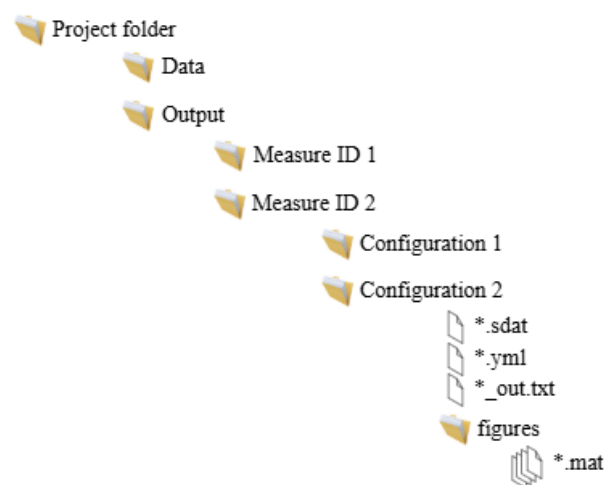
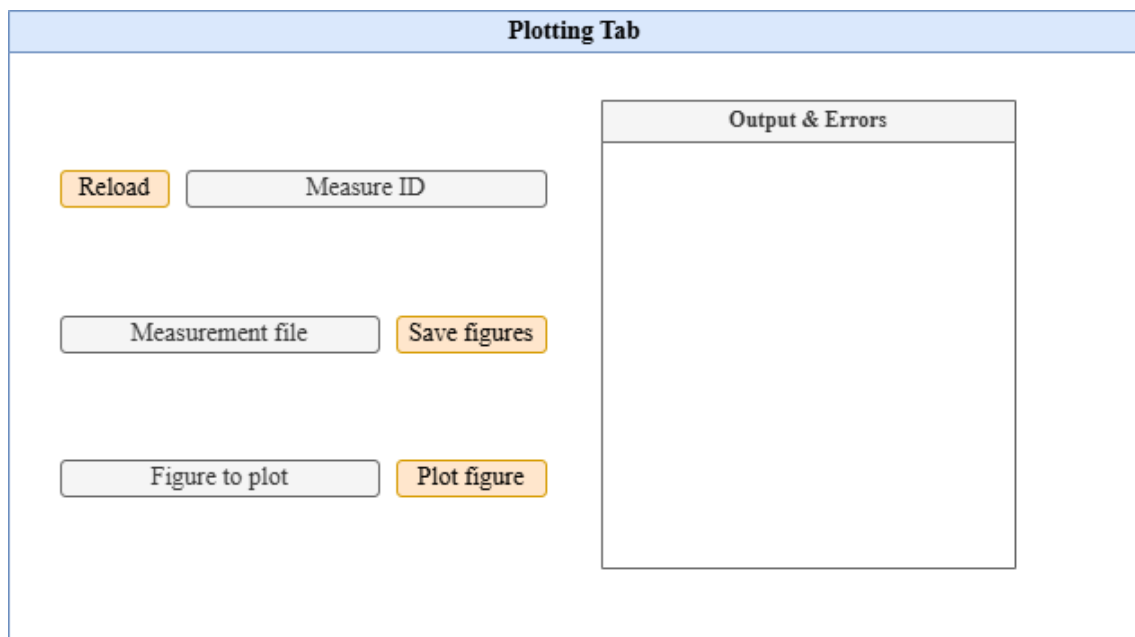


Figure 15: Project folders organisation

GRASP execution output is saved in a file named `UPC_{config}_out.txt`. This file will be the one used in order to create and save all the different files for each figure. Therefore, it is necessary first to specify the measure ID, then the measurement file and finally, the figure name that is desired to be seen.

These actions can be implemented with the different buttons and text inputs shown in Figure 16. Additionally, since this tab will also be executing Matlab scripts, it will be helpful for the user to see the progress and possible errors that can appear.



The figure shows a UI design for a 'Plotting Tab'. It features a light blue header bar with the text 'Plotting Tab'. Below this, the interface is divided into two main sections. On the left, there are three rows of controls: the first row has an orange 'Reload' button and a grey 'Measure ID' text input; the second row has a grey 'Measurement file' text input and an orange 'Save figures' button; the third row has a grey 'Figure to plot' text input and an orange 'Plot figure' button. On the right, there is a large rectangular area with a grey header 'Output & Errors' and a white body, intended for displaying logs or error messages.

Figure 16: Plot tab design

8 Implementation

The code implemented for this project can be found in the GitHub repository[22]. Some implementations have been explained previously in other sections of this document like the Matlab Controller or the Aeronet Download Service, but this section aims to explain the purpose of the different classes and the different design patterns that have been used in order to follow the SOLID principles as much as possible.

8.1 Set up development environment

In order to create this project, it is necessary to meet several requirements to have a correct development environment. First of all, it is necessary to choose a correct IDE. There are different options, including JetBrains Rider and Visual Studio Code, but for this project, it has been decided to use Visual Studio Community [17].

The application will be run using .NET, which is a free and open-source application platform supported by Microsoft used for developing different type of applications using C# [19] and which is usually already installed in the system[18].

In order to use Avalonia, it is necessary to install the corresponding extension for Visual Studio. It can be done by accessing to Extensions\ManageExtensions:

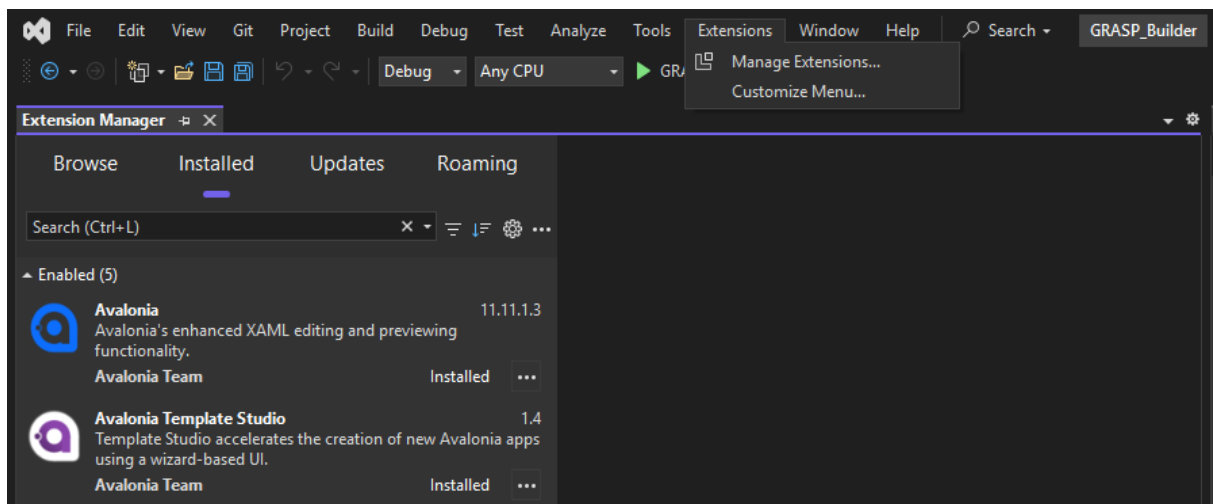


Figure 17: Extensions installed in Visual Studio

Since the app will be executing Matlab scripts, it is also necessary to have Matlab installed. In this case it has been used Matlab R2024a[20].

In order to run the GRASP algorithm it is necessary to access to CALCULA, where it is already installed and running in Ubuntu that will only be used for testing.

8.2 Views and ViewModels

In order to not overload the content of the different files for the User Interface implementation (Views and Windows) and management (View Models) it has been created a different View class for each Tab and for each Window, that can be resumed in the following list:

- About Window: Window that shows general information about the app and containing button "Help" which opens User Manual
- Data Combination View: Tab where the user can select Measure ID to combine data, showing available configurations for selected date and time and executing GRASP.
- Data Download View: Tab where the user can select dates and station and downloading corresponding data from Earlinet and Aeronet websites.
- Home View: Shows the following action options: to create, open or import a project.
- Home Project Action Window: Window that appears when the user clicks on a project action option in Home View and in charge of performing it.
- Main Window: Window that contains all the tabs and shows them when necessary. It can show the Home View when no project is loaded or a combination of the other tabs with an additional Log and a Menu in order to manage the app Settings or the current project.
- Message Window: Window that shows a message to the user that can be an Error, a Warning or just information.
- Log View: Tab that shows the Log content of the app.
- Plot View: Tab where the user can select and create the desired figures after executing GRASP algorithm.
- Settings View: Window where the user is able to modify the app configuration parameters.

All these classes for both Views and ViewModels are implemented using the MVVM pattern using ObservableObject and RelayCommand classes from the .NET community toolkit. In addition, all ViewModels classes have been structured by regions in order to homogeneously group the different properties, commands and methods.

In order to establish communication between different classes to notify modifications, it has been used the Messenger class from the .NET community toolkit. It consists on defining a Send Action, that can include some parameter or variable and a Receive Action, that will be executed when the Send Action is called, executing the linked method.

In the following example it can be seen how the DataCombinationViewModel class has been implemented using the Messenger class to receive modifications in the buttons enabled state:

```
1 public DataCombinationViewModel()  
2 {
```

```
3     Messenger.Default.Register<bool>("UpdateButtonsEnabled",
4         UpdateButtonsEnabled);
5 }
6 private void UpdateButtonsEnabled(bool status)
7 {
8     AreButtonsEnabled = status;
9     if (!status)
10    {
11        isEnabled_D1_L = false;
12        isEnabled_D1P_L = false;
13        isEnabled_D1_L_VD = false;
14        isEnabled_D1P_L_VD = false;
15    }
16 }
```

In the corresponding controller, in order to send the modification in order to enable the buttons, for example, it is necessary to execute the following line:

```
1 Messenger.Default.Send<bool>("UpdateButtonsEnabled", true);
```

Using Messenger allows to notify modifications in the App between classes that exist simultaneously, in this case between the different Tabs and between ViewModels and Controllers. In the case of creating new Windows like the Settings one, it is not necessary, since the needed information can be passed as parameters to the Constructor when it is initialized.

8.3 Controllers

In the backend of the app, different classes and controllers have been implemented following the Single Responsibility Principle (SRP). Each one has its own responsibility and purpose:

- Download Controllers: Different controllers have been developed for each different web site. They both implement the IDownloadController Interface. Each controller is responsible of the different actions that need to be executed when downloading the respective data by managing their respective web services.
- CmdController: This controller is responsible of the different actions needed to execute commands, in this case, necessary to execute the GRASP algorithm via the command line. Since this application have been designed to work with CALCULA, command line for executing GRASP only is executed with Ubuntu sintaxis.
- AppConfig: This class manages the application configuration, obtaining and saving the different configuration parameters from the configuration file (config.json), located in the execution directory.
- Helpers: Different helper classes have been developed to perform different actions. FormatHelpers are used for changing the format of variables, for exemple: Converting a dictionary to a string variable. FileHelpers are used for managing the files and directories renaming, copying, etc.

- **Logger:** This class is responsible of managing the logging actions.
- **MessagesController:** This controller is responsible of managing the different messages that can be shown to the user: Errors, warnings and information messages.
- **ProjectConfig:** This class manages the project configuration, obtaining and saving the different configuration parameters from the project file (project.grasp), located in the project directory.
- **MatlabScriptController:** This controller is responsible of managing the different actions needed to execute any MatlabScript.

8.4 Polymorphism through interfaces

With the objective of applying the Open/Closed Principle (OCP) and the Interface Segregation Principle (ISP), different interfaces have been implemented in order to be used in different points of the project. Using polymorphism, enables the possibility of using objects of a derived class as objects of a base class[23].

Polymorphism can be implemented with a parent or base class, which defines the common properties and implements virtual methods. Derived classes can use the already implemented methods of the parent class, or override them.

Another possible implementation of polymorphism is through the use of abstract classes or interfaces, which is the one used in this project. Interfaces are contracts that define a set of methods that a class must implement. This enables the possibility to work with different classes, where each class implements the interface in a different way but all classes and actions are used uniformly by calling the interface methods.

8.4.1 Factory pattern

Factory pattern is a creational design pattern that provides an interface for creating objects of a parent class, but deciding which subclass to instantiate[24]. This allows to resume all the different conditions in one single method, instead of using multiple if-else conditions during the development of the code.

An exemple of this pattern can be seen in the following example, showing the implementation of the DownloadControllerFactory Create method:

```
1 public static IDownloadController Create(DownloadType type, string
   _repositoryDirectory, string _workingDirectory)
2 {
3     switch (type)
4     {
5         case DownloadType.Earlinet:
6             return new EarlinetDownloadController(_repositoryDirectory,
               _workingDirectory);
7         case DownloadType.Aeronet:
8             return new AeronetDownloadController(_repositoryDirectory,
               _workingDirectory);
9         default:
10            throw new NotSupportedException($"Download type {type}.
               ToString() is not supported");
```

```
11     }  
12 }
```

8.4.2 Project Actions

The first usage of this coding strategy can be found in `IProjectAction` interface. This interface is used when initializing a project when the application is opened, where the user can choose between the different actions that can be executed: Create, Open or Import from .zip file. Export action has also been implemented, but it is also accessible through the File Menu, once a project is loaded.

```
1 public interface IProjectAction  
2 {  
3     public string Title { get; set; }  
4     public string ProjectName { get; set; }  
5     public string DirectoryPath { get; set; }  
6     public bool IsProjectNameVisible { get; set; }  
7     public bool IsDirectoryPathVisible { get; set; }  
8     public Task<bool> Execute();  
9     public Task Browse();  
10 }
```

This interface defines five different properties, that are used for initializing the View and ViewModel before showing the new window, and two methods, that correspond to the different actions that can be executed in this window: Browse a directory and Execute.

With this implementation, `IProjectAction` object is used in `HomeProjectActionView-Model`. Only in the constructor the type of action needs to be specified. In the rest of the class, all the Bindings and Commands are executed using this object equally for all the types. A simple example is the implementation of Execute command.

```
1 public ICommand BrowseCmd => new RelayCommand(async _ => await  
    BrowseExecute(), CanBrowse);  
2  
3 private bool CanBrowse(object _)  
4 {  
5     return true;  
6 }  
7  
8 private async Task BrowseExecute()  
9 {  
10     _projectAction.DirectoryPath = DirectoryPath;  
11     _projectAction.ProjectName = ProjectName;  
12  
13     await _projectAction.Browse();  
14  
15     DirectoryPath = _projectAction.DirectoryPath;  
16     ProjectName = _projectAction.ProjectName;  
17 }
```


In this Command, it can be appreciated the simplicity of the code and the lack of if-else conditions and knowledge of the type of action that is being executed.

8.4.3 Download Controllers

Another class that can take advantage of the use of polymorphism is the DownloadController, which needs a simpler interface.

```
1 public interface IDownloadController
2 {
3     public string RepositoryDirectory { get; set; }
4     public string WorkingDirectory { get; set; }
5     public Task Download(DateTime FromDate, DateTime ToDate, string
6         station);
7 }
```

With this implementation, the only method that needs to be called is Download, but in each implementation, this method execute different actions.

For the moment, only two classes are implemented using this interface, but in the future, if it was desired to add any other data source, it would be possible to implement a new class that implements this interface and used it in the ViewModel as it is done now with the IDownloadController object.

```
1 IDownloadController downloadController = DownloadControllerFactory.
    Create(DownloadType.Earlinet, _earlinetRepositoryDirectory,
        _workingDirectory);
2 await downloadController.Download(FromDate, ToDate, "BRC");
```

8.4.4 Matlab Scripts Implementations

The last interface that has been implemented is IMatlabScript. Running Matlab scripts using C# is not a complicated task, since the use of the library ProcessStartInfo agilizes all the process, needed only the name of the script to be executed. But for this project it was required to execute more actions.

```
1 public interface IMatlabScript
2 {
3     public string Name { get; }
4     public Dictionary<string, object> vars { get; set; }
5     public void PreExecutionActions();
6     public void PostExecutionActions(bool resultOK = true);
7 }
```

It was needed not only to run a script but to have communication between the Matlab scripts and the application. In order to solve this problem, it was decided to create different files:

- Configuration files: Parameters are saved from C# application and read in Matlab before running the corresponding script.

- Output files: Results are saved from Matlab and read in C# application after running the corresponding script, in order to actualize the UI or execute further actions.

This is very useful, since it allows to have a clear separation between the different tasks and to make the code more maintainable. If it is desired to add a new script, it is only necessary to create a new class using this same interface and call the method for running a Matlab script as in the following example:

```
1 MatlabController.RunMatlabScript(ScriptType.Preview, dict, "_DC");
```

This is possible because this interface is only used for running Matlab Scripts. The `MatlabController` class is responsible of executing all the sequence for running a script, which consists on:

1. Execute pre-execution actions
2. Run the script (start matlab process)
3. Receive output from Matlab
4. Receive results from Matlab
5. Execute post-execution actions

Each Script implementation will be the responsible of defining the different actions to be executed in each step, having into account if the script finished with error or was executed successfully.

8.5 Interactions with other applications

The main goal of this application is to create a user-friendly interface for executing all the sequence of actions needed to download data from different web services, combine it and create plots from the results. In order to do that, the application needs to interact with different external applications and web services:

1. Web services: in order to download all the necessary data files from the different sources, Aeronet and Earlinet sites
2. Matlab: in order to execute the different scripts needed for data combination and plotting
3. GRASP algorithm: in order to execute the algorithm and obtain the results

With the use of the different interfaces and classes explained in this section, it is possible obtain the desired behaviour, which can be summarized in the following diagram:

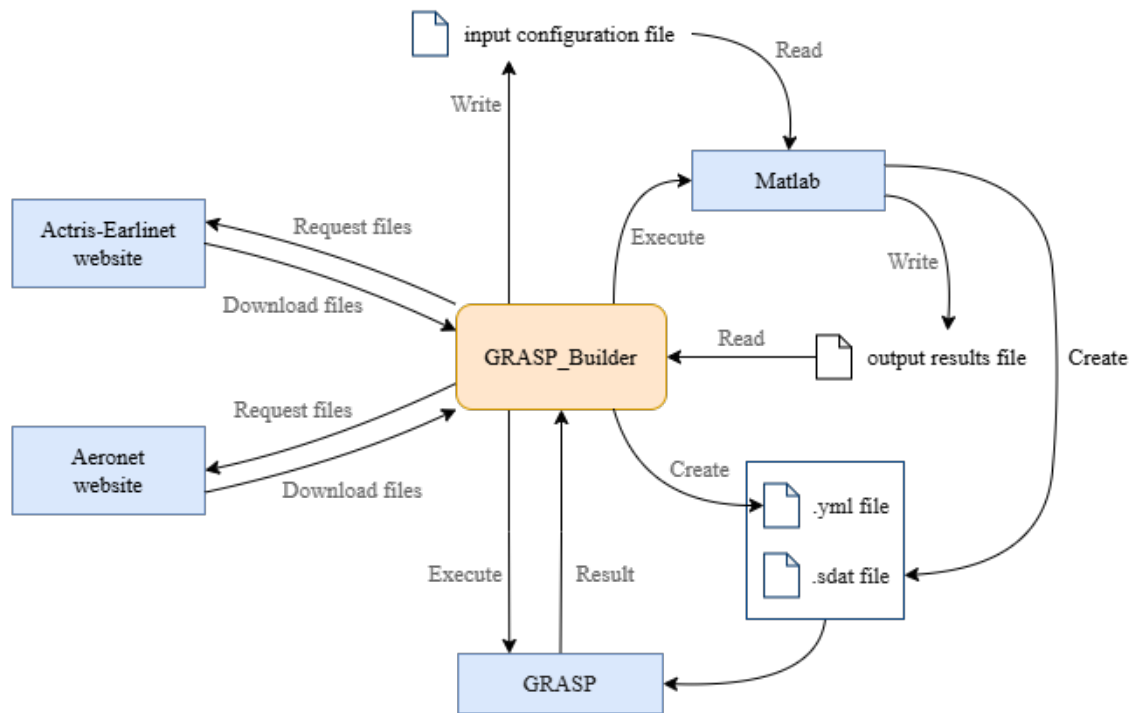


Figure 18: Application interactions diagram

In Figure 18 it can be appreciated all the different actions that are taking place during the execution of the application and the communication between the application and the external applications and web services. All these interactions are not noticed from the user point of view, giving the impression of a easier user experience.

9 Testing

During the development of the application, different tests have been performed to ensure that the application works as expected:

1. First contact:

In order to know how the initial implementation was working and optimize the process it was needed to know how the different steps were being executed and the expected results

- Download files
- Execute Matlab scripts with old data
- Execute GRASP with old data

2. Test downloaded data: After automatizing the download process, it was needed to test the different matlab scripts with the new data. During this tests, it was detected that some data was being downloaded with a new file format. This was a problem, since the old scripts were not compatible with this new format and they had to be modified.
3. Test GRASP with generated data: After modifying the scripts, it was needed to test the obtained data with the GRASP algorithm. It was detected that the data was not being processed correctly since the configuration file did not match the correct values. This was an issue with file versions that was solved by updating the configuration file to the correct version.
4. Test full UI execution: After defining the application as configurable and workspace independent, it was needed to test the full execution of the application. During this round of tests, it was checked that the project folders were working as expected, improving the project management process.
5. Test full UI execution in CALCULA: Since CALCULA works with LINUX different aspects of the application had to be tested. Some changes were needed for the folders and files management, since some routing was only working in Windows. During this tests the full execution of the application was performed successfully, since it was the only test where the GRASP algorithm could be executed.

Unit testing has also been performed to ensure that the application is working as expected. Unit testing consists in testing the separated components of the application, in this case the Model classes, to ensure that they are working as expected[26].

Implementing this type of testing in a project allows to detect and fix some issues that were not detected during the integration tests and reduces the time consumption of some testing process.

Test	Duration	Traits	Error Message
UnitTests (13)	2,3 sec		
UnitTests (13)	2,3 sec		
ExecuteMatlab (4)	652 ms		
MatlabController	2 ms		
MatlabScriptFactory_CreatesEx...	650 ms		
SaveFigures	< 1 ms		
SendFiles	< 1 ms		
ManageConfigurations (6)	839 ms		
LoadAppConfig	86 ms		
LoadProjectConfig	15 ms		
ModifyAppConfig	4 ms		
ModifyProjectConfig	20 ms		
SaveAppConfig	18 ms		
SaveProjectConfig	696 ms		
ManageProjects (3)	763 ms		
CreateProject	35 ms		
ImportProjectData	722 ms		
OpenProject	6 ms		

Figure 19: Implemented Unit Tests

In 19 it can be seen the implemented unit tests for this project, where different classes have been tested: application configuration, project configuration and Matlab script management. The main advantages of this type of testing are that they can be performed efficiently, in this case they have been executed within seconds, and can be performed during development. This helps the developer to check that the new code is still working as expected.

10 Results

In this section it is described the released version of the application and the final execution sequence, even a more detailed explanation can be found in the User Manual, in the Annex.

10.1 Final User Interface

During the development of the application, the user interface has been modified to be more user-friendly and to add more features that were not contemplated during the design phase.

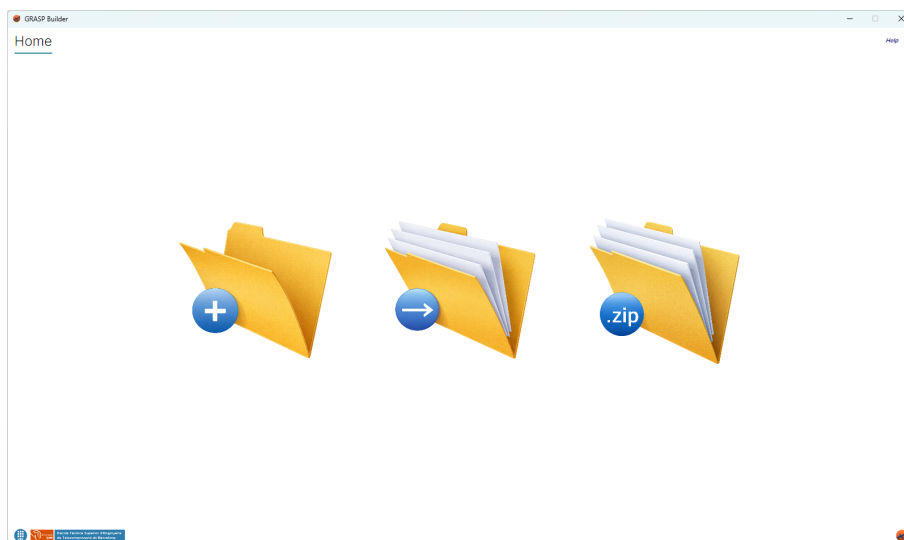


Figure 20: Home view

The Home view, which is shown when the application is initialized, allows the user to select the different options available in the application: Create, Open or Import project. This can be seen in Figure 20. It has also been added the help button, which opens the User Manual in .pdf format.

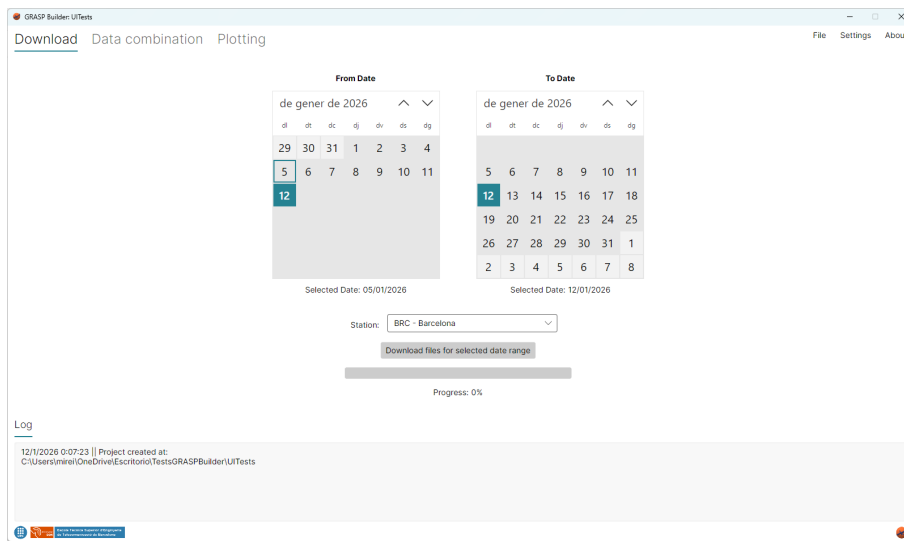


Figure 21: Download view

The Download View maintains mostly the initial design. Since the process of downloading files can depend on the user's internet connection, it has been added a progress bar to show the progress of the download.

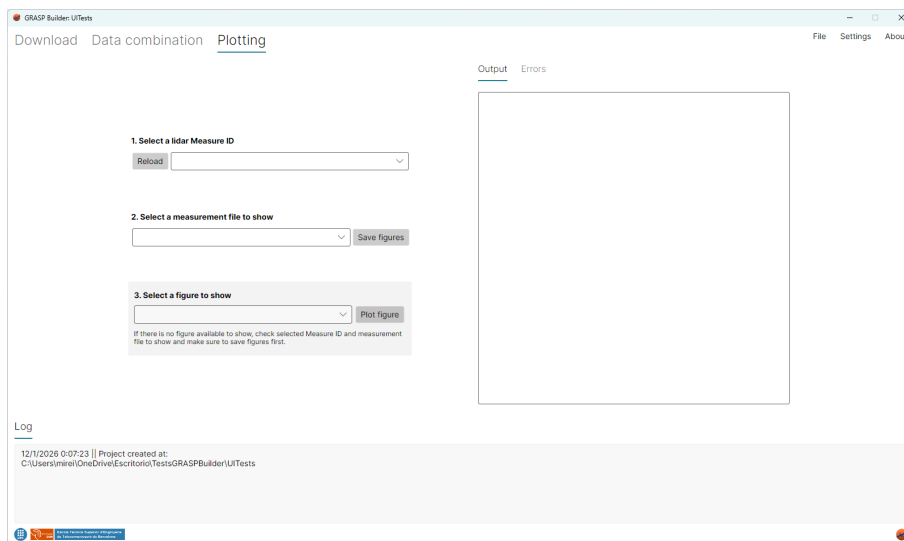


Figure 22: Data Combination view

Data Combination has not been modified from the initial design, since it allows the user to select the different files to be combined and the different parameters to be used for the combination. This can be seen in Figure 28.

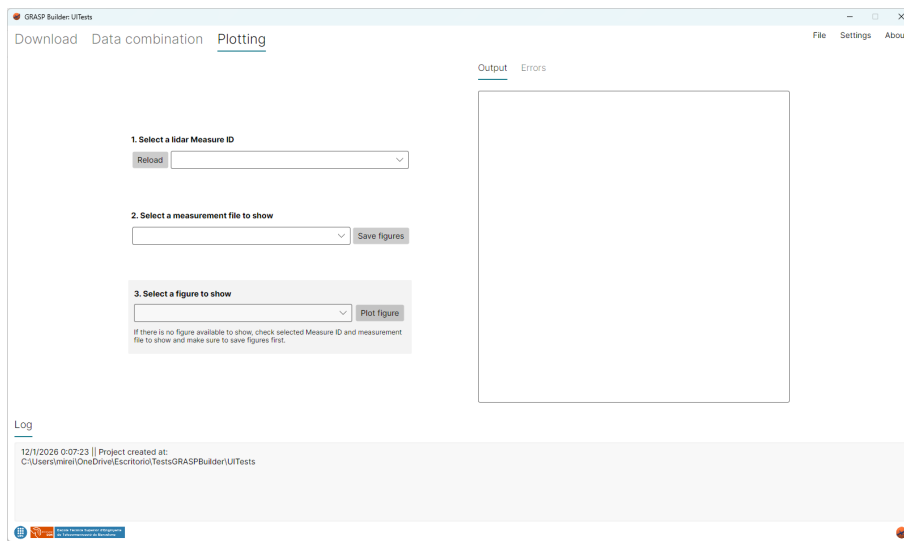


Figure 23: Plotting view

Plotting view has not been modified from the initial design neither, since it allows the user to select the different files to be plotted and the different parameters to be used for the plotting, shown in Figure 29.

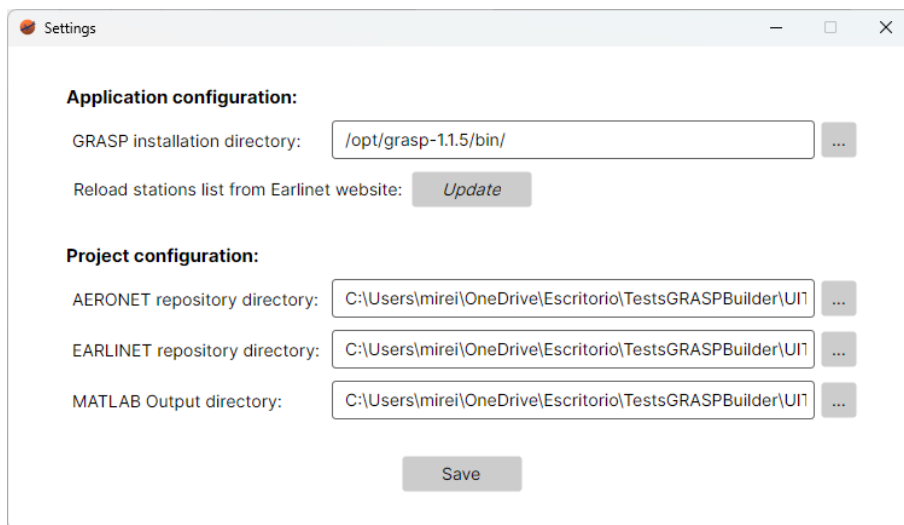


Figure 24: Settings window

Settings window has been created in order to allow the user to configure both the application and project settings, shown in Figure 24. This type of configuration has been clearly separated, in order to let the user know which modifications will affect only the project and which ones will affect all executions of the application, independently of the project loaded.

10.2 Work-flux

The final work-flux for working with GRASP algorithm can be resumed in the following steps, all included in the developed application and executed in CALCULA environment:

1. Download files: Select date range and station and download the corresponding files. Available files will be shown in same View
2. Combine data: Select the desired MeasureID, choosing the data and time range to be used. Heights can be configured and available configurations will be automatically shown.
3. Execute GRASP: Send data will create the input files automatically for GRASP algorithm and will execute it.
4. Plot results: Available results will be automatically detected. If possible, list of figures will be created and saved in Output directory. The user will be able to choose which figure to show.

10.3 Low resolution User Interface

Using Avalonia provokes the inability to resize the application window, since it does not maintain the correct format for the moment. In some tests, it has been detected that all actions can not be executed in a low resolution system. Then, it has been created the option of a low resolution UI, which can be selected by right clicking the application window and selecting the option "Change to Low resolution UI", shown in Figure 25.

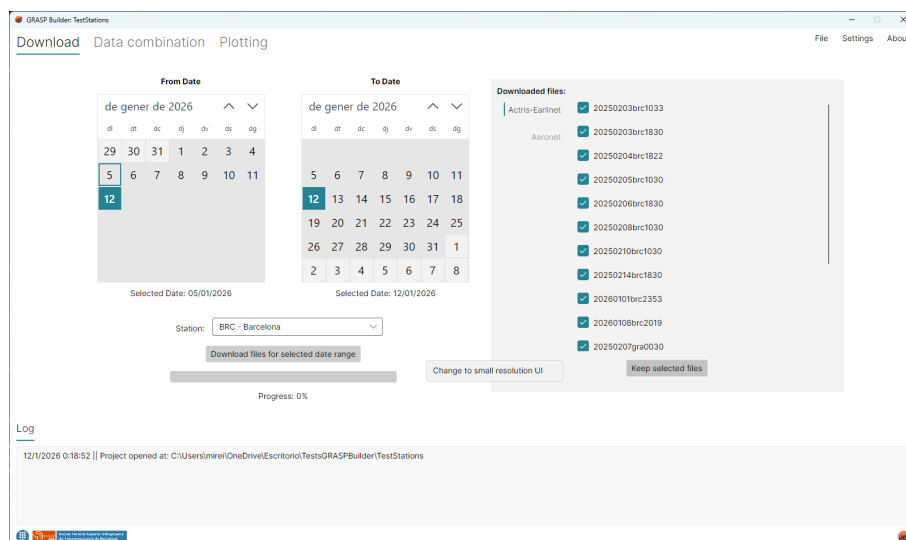


Figure 25: Low resolution UI

This option allows the user to execute the application in a low resolution system, having a slightly modified UI, which can be seen in the following figures: Figure 25

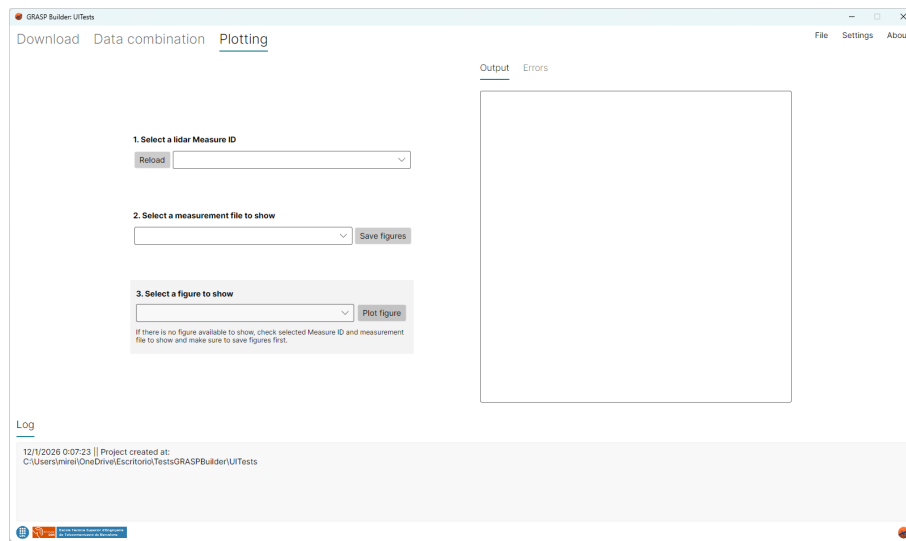


Figure 26: Plotting view

Plotting view has not been modified from the initial design neither, since it allows the user to select the different files to be plotted and the different parameters to be used for the plotting, shown in Figure 29.

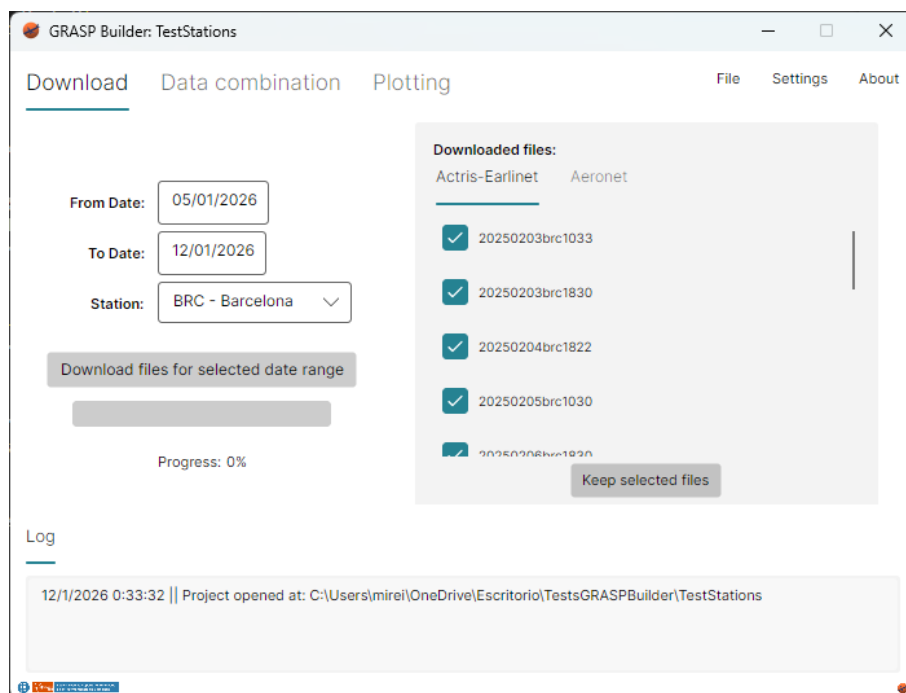


Figure 27: Download view for low resolution UI

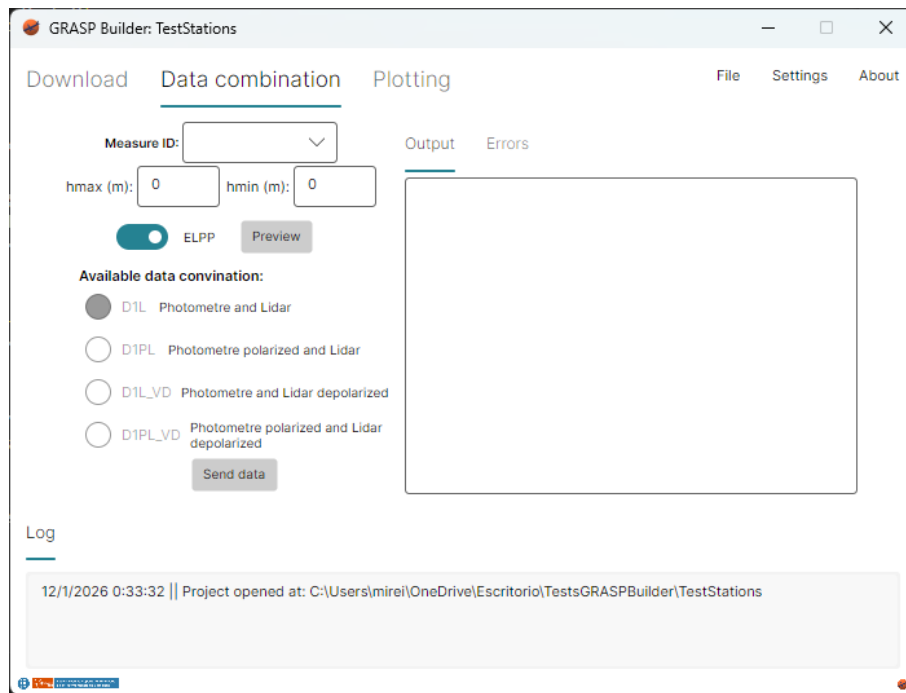


Figure 28: Data Combination view for low resolution UI

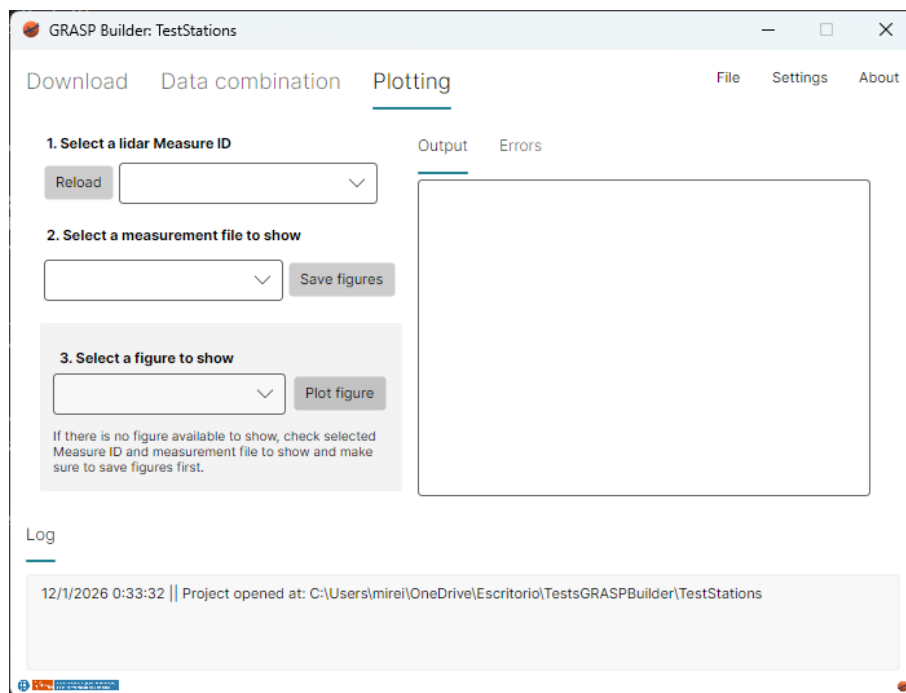


Figure 29: Plotting view for low resolution UI

This option allows the user to execute the application in a low resolution system, since this User Interface is smaller than the lowest resolution option.

11 Conclusions

References

- [1] Wolfgang Skala. Drawing gantt charts in latex with tikz.
- [2] Albert Einstein. Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies]. *Annalen der Physik*, 322(10):891–921, 1905.
- [3] Michel Goossens, Frank Mittelbach, and Alexander Samarin. *The L^AT_EX Companion*. Addison-Wesley, Reading, Massachusetts, 1993.
- [4] Donald Knuth. Knuth: Computers and typesetting.
- [5] C.N.R. IMAA. Actris-earlinet data portal. Web Application available at <https://data.earlinet.org/earlinet/desktop.zul>, 2024.
- [6] NASA. Aerosol robotic network (aeronet). Web Application available at <https://data.earlinet.org/earlinet/desktop.zul>, 2025.
- [7] Miriam Martínez Canelo. ¿qué es la programación orientada a objetos? Web Application available at <https://profile.es/blog/que-es-la-programacion-orientada-a-objetos/>, 2025.
- [8] MathWorks. Matlab. Web Application available at <https://es.mathworks.com/products/matlab.html>, 2025.
- [9] C.N.R. IMAA. Actris-earlinet rest api. Web Application available at <https://data.earlinet.org/api/swagger-ui/>, 2025.
- [10] NASA. Aerosol optical depth. Web Application available at https://aeronet.gsfc.nasa.gov/print_web_data_help_v3_new.html, 2025.
- [11] NASA. Raw products aerosol optical depth. Web Application available at https://aeronet.gsfc.nasa.gov/print_web_data_help_v3_raw_sky_new.html, 2025.
- [12] NASA. Aerosol inversions. Web Application available at https://aeronet.gsfc.nasa.gov/print_web_data_help_v3_inv_new.html, 2025.
- [13] Microsoft. Mvvm. Web Application available at <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>, 2025.
- [14] Microsoft. Observableobject. Web Application available at <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/observableobject>, 2025.
- [15] Microsoft. RelayCommand. Web Application available at <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/relaycommand>, 2025.
- [16] GeeksforGeeks. Solid principles. Web Application available at <https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/>, 2025.

-
- [17] Microsoft. Download visual studio community. Web Application available at <https://visualstudio.microsoft.com/es/vs/community/>, 2025.
 - [18] Microsoft. Download .net. Web Application available at <https://dotnet.microsoft.com/en-us/download/dotnet>, 2025.
 - [19] Microsoft. What is .net? Web Application available at <https://dotnet.microsoft.com/en-us/learn/dotnet/what-is-dotnet>, 2025.
 - [20] MathWorks. Download matlab. Web Application available at <https://es.mathworks.com/help/install/ug/install-products-with-internet-connection.html>, 2025.
 - [21] CALCULA. Calcula. Web Application available at <https://ondemand.tsc.upc.edu/pun/sys/dashboard>, 2025.
 - [22] GitHub. Github. Web Application available at <https://github.com/mireialopez0910/TFM/tree/main>, 2025.
 - [23] Microsoft Learn. Polymorphism. Web Application available at <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism>, 2025.
 - [24] Microsoft Learn. Factory pattern. Web Application available at <https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/>, 2025.
 - [25] GRASP Open. Grasp. Web Application available at <https://www.grasp-open.com/>, 2025.
 - [26] AWS. Unit testing. Web Application available at <https://aws.amazon.com/es/what-is/unit-testing/>, 2025.

Appendices

Appendices may be included in your thesis but it is not a requirement.